Verifiable and Lightweight Multi-Round Secure Federated Learning

Jiayin Li, Shengmin Xu, *Member, IEEE*, Xingshuo Han, Jianting Ning, *Member, IEEE*, Xinlei He, *Associate Member, IEEE*, and Guowen Xu, *Senior Member, IEEE*

***Abstract*—Federated learning (FL) is a paradigm that ensures the confidentiality and accessibility of data without requiring the collection of private data from multiple sources. It acquires an aggregation model by integrating various local models from clients. However, clients are vulnerable to numerous security and privacy threats. Existing solutions were unable to implement training models that are both dropout-resilient and lightweight while also providing verification capabilities when large-scale clients are involved in federated training. To improve the usability of FL, we propose a verifiable and lightweight multi-round secure FL framework by designing and incorporating a double-masking mechanism to ensure secure transmission. Moreover, we optimize the secure aggregation strategy by designing a dropout-resilience method via the secret-sharing mechanism. Specifically, we establish a lightweight model-secure training scheme and provide a parameter reuse strategy by constructing a full connection graph, which reduces computational cost and communication overhead. Furthermore, we propose a secure authentication protocol that enables the client to verify the accuracy of the computing results from the server. Extensive experimental evaluations indicate that our solution demonstrates relatively modest performance but superior functionality compared to current state-of-the-art methods. In particular, we can achieve the verification function with an acceptable increase in computational cost of approximately 200 ms per epoch.**

***Index Terms*—Federated learning (FL), privacy protection, aggregation verification, model security.**

I. Introduction

FEDERATED learning (FL) is a distributed learning paradigm that has been widely used in current banking and medical institutions. It helps to improve decisions without requiring institutions to share their private data in areas such as financial risk assessment, financial investments, and disease diagnosis [1], [2], [3]. FL prevents privacy leakage during data outsourcing and enables those lacking training data to participate in developing robust models to avoid isolated data. Simultaneously, collaborative training through FL has become the primary approach for leveraging the processing power of many smart devices. According to the report from FL-IJCAI'2023 [4], FL was initially launched by Google in 2016 and was first innovatively applied in the banking industry in 2018. The field of FL has since experienced rapid expansion. Consequently, there has been a growing amount of research on applying FL in the publications of various prestigious organizations, including FL aggregation (FLA) [5], [6], [7], trusted FL (TFL) [8], [9], [10], [11], and intellectual property protection (IPR) [12], [13], [14].

Although FL eliminates the potential for privacy leakage resulting from data outsourcing, it still faces some risks, including secure gradient aggregation [15], [16], model privacy [17], [18], [19], poisoning attacks [20], [21], and so on. To address the concern for privacy-preserving FL, various techniques have been proposed to ensure the security and reliability of the entire FL framework, including the use of homomorphic encryption [22], multi-party secure computation [23], and differential privacy [24]. Fig. 1 depicts the process of central deep learning training and privacy-preserving FL training, respectively. Regarding the privacy-preserving FL system shown in Fig. 1(b), it mainly contains three entities: (1) the key generator is a trusted authority with the responsibility of generating and sending the public/private key; (2) the clients are regarded as

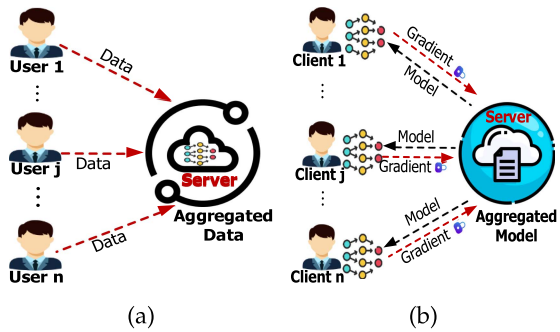


Fig. 1. (a) Aggregation data from different sources for central training. (b) Privacy-preserving FL framework.

Received 1 November 2024; revised 26 August 2025; accepted 27 August 2025. Date of publication 4 September 2025; date of current version 14 January 2026. This work was supported in part by the National Key R&D Program of China under Grant 2023YFB3106200, in part by the Natural Science Foundation of Fujian Province under Grant 2023J05128, and in part by the National Natural Science Foundation of China under Grant 62372108, Grant 62471139, Grant 62402109, Grant 62425205, Grant 62502075, and Grant 62572123. *(Corresponding author: Shengmin Xu.)*

Jiayin Li and Shengmin Xu are with the College of Computer and Cyber Security, Fujian Normal University, Fuzhou, Fujian 350117, China (e-mail: lijiayin2019@gmail.com; smxu1989@gmail.com).

Xingshuo Han is with the College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics, Nanjing 210016, China (e-mail: xingshuo.han@nuaa.edu.cn).

Jianting Ning is with the School of Cyber Science and Engineering, Wuhan University, Wuhan 430072, China, and also with the Faculty of Data Science, City University of Macau, Macau 999078, China (e-mail: jtning88@gmail.com).

Xinlei He is with the Hong Kong University of Science and Technology, Guangzhou 999077, China (e-mail: xinleihe@hkust-gz.edu.cn).

Guowen Xu is with the School of Computer Science and Engineering, University of Electronic Science and Technology of China, Chengdu 611731, China (e-mail: guowen.xu@foxmail.com).

Digital Object Identifier 10.1109/TDSC.2025.3606413

1545-5971 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Anhui University. Downloaded on October 13,2026 at 09:40:23 UTC from IEEE Xplore. Restrictions apply.

data owners who train the local models and submit the encrypted gradients to the server; (3) the server is tasked with initializing the global model and aggregating local gradients. While privacy-preserving FL systems enhance data and model security, they face substantial obstacles concerning efficiency, lightweight design, and dropout.

Besides, privacy-preserving FL techniques have been extensively researched. Nevertheless, when multiple clients are in distributed training, existing methods fail to meet the following practical requirements:

Client dropout-resilience: A framework for privacy-preserving FL is implemented to create a highly secure global model by aggregating the gradients from several clients. Unfortunately, due to the limitations imposed by utilising the client's computer capabilities, battery power, and ad hoc tasks, the training process encounters an unstable connection between the client and the server, which triggers a temporary client dropout. It's challenging to achieve flexible aggregation throughout the entire training process.

Lightweight training: Asymmetric encryption tools are essential for privacy-preserving FL. However, when clients drop out, the server is unable to obtain the appropriate key and therefore cannot perform the aggregation computation. To address the above issue, the system must generate the required public/private keys before performing each round of training, which significantly increases the computational cost of the system. Therefore, a more effective privacy-preserving FL framework should support lightweight model training and multiple rounds of secure key reuse.

Verifiable returned results: The server undertakes the computation of vast local gradient aggregations and submits the aggregation model to each client. However, the system lacks the assurance that the server is fully performing trusted aggregation operations, making it impossible for clients to confirm that the received aggregation model is delivered accurately. To determine whether the server has obtained the aggregation results by correctly performing the relevant operations, the client must be able to verify the results returned by the server.

To our knowledge, existing solutions, such as asynchronous federated learning and Byzantine fault tolerance mechanisms, primarily focus on system robustness, which generally assume the central server is honest and do not verify the correctness of aggregated results. Additionally, when handling multi-round federated secure aggregation, repeated configurations lead to a significant increase in system overhead, making it challenging to achieve lightweight federated secure training and address the issue of client dropout behavior. *Is it feasible to design a privacy-preserving FL framework with client dropout-resilient capabilities, lightweight federated training, and verifiable aggregation results via leveraging the general construction of cryptographic tools to better adapt to complex multi-round federated secure aggregation environments?* This standard is fundamental to the evaluation of the stability, reliability, and efficiency of FL. Therefore, the design of a secure, lightweight, and resilient FL framework with a verification function represents a significant challenge that requires immediate attention.

A. Contributions

In this paper, we build a construction of a dropout-resilient, lightweight, and verifiable multi-round secure FL framework. By employing cryptographic techniques, we establish a robust and reliable FL aggregation approach that effectively addresses the issue of client dropouts and verification. In summary, this paper makes significant contributions to the following:

- To strengthen the usability of federated training, we propose a framework to enhance the efficacy of multi-round FL aggregation for a substantial number of participating clients. This framework effectively eliminates the potential for inaccurate aggregation results.
- To enhance both security and computational efficiency in local model sharing, we optimise a lightweight security-sharing protocol based on secret sharing and double masking. Specifically, a privacy-preserving FL framework is established using a server architecture for the aggregation of client gradients. This technique is executed without deciphering the gradients of the clients, thereby eliminating the impact of offline clients on the aggregation output.
- To validate the aggregation results from the server, we employ a homomorphic hash function in combination with pseudorandom technology, which serves as the foundational framework for our method. This approach allows clients to obtain the benign aggregation model from the server.
- Finally, we present a comprehensive security analysis of the proposed framework, which encompasses numerous clients. This security solution remains effective even in the event of collusion between the server and clients. Furthermore, the effectiveness and security of the proposed method are evaluated through image classification on real-world datasets.

II. Background and Related Work

According to research on existing works, a significant number of privacy-preserving techniques for FL have been proposed to address the issues above [25], [26], [27]. To demonstrate the novelty of the proposed approach, we compare the results between theirs and ours, as shown in Table I. Table I illustrates that the existing solutions are ineffective in addressing the three major challenges outlined above and are not conducive to full-scale promotion and deployment.

Previous research has mainly relied on homomorphic encryption (HE), multi-party computing (MPC), and Shamir's secret sharing (SS) to develop efficient solutions and achieve deep learning training while ensuring privacy in FL. Fan et al. [28] proposed a CKKS method for achieving homomorphic encryption, but it suffered from significant inefficiencies in both communication and processing. Afterwards, Kim et al. [31] proposed a secure method for securing FL training and prediction data using a logistic regression model with additive homomorphic encryption based on integers as plaintext. Likewise, Phong et al. [32] introduced a way to preserve the privacy of the global model by allowing participants to possess the decryption

Authorized licensed use limited to: Anhui University. Downloaded on October 13,2026 at 09:40:23 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON OF REPRESENTATIVE RELATED SOLUTIONS

	Privacy Protection	Multi-Clients Participation	Lightweight Training	Dropout Resilience	Keys Reuse	Multi-Round Execution	Aggregation Verification
FV12 [28]	●	○	○	○	○	○	○
BBG+20 [29]	●	●	◐	○	○	○	○
XLL+20 [30]	●	●	◕	●	○	○	●
HSBT22 [23]	●	●	◕	●	○	○	○
MWA+23 [7]	●	●	●	●	●	●	○
Ours	●	●	●	●	●	●	●

Ours: Our Method in Table 1. ○: No. ●: Yes. ◕ and ◐: Partially.

key. Notably, Stacey et al. [33] proposed a solution for robust privacy-preserving FL with satisfactory model performance. Their solution utilises Paillier homomorphic encryption as the basic technique. However, the HE technology offers robust privacy protection, although its effectiveness makes it challenging to implement in resource-constrained contexts. To enhance the effectiveness of secure training for privacy-preserving FL, Mohassel et al. [34] presented the SecureML framework, which utilised secure two-party computation. This enabled data owners to encrypt, process, and secretly distribute their data across two non-colluding servers. Chaudhari et al. [35] proposed a secure framework based on a four-party MPC, which reduces the number of active participants and enhances the efficiency of online communication. Furthermore, Bonawitz et al. [36] introduced a secure aggregation protocol that employed both SS and authenticated encryption. Liu et al. [37] put forth a privacy-preserving k-means technique to facilitate proactive caching in cellular networks based on FL and SS. However, the research above was exclusively concerned with the security of local gradients in FL and failed to address the pivotal challenges that arise during the federated aggregation phase.

Therefore, some work [7], [29], [38] proposed secure aggregation specifically tailored to FL scenarios recently. For example, Bell et al. [29] introduced the BBGLR system, a pioneering protocol designed to facilitate the management of aggregation with thousands of clients and high-dimensional input vectors while enabling devices to drop out at any stage of their execution. To address the vulnerability of traditional federal learning to attacks, Fotohi et al. [39] proposed a secure blockchain-enabled federated deep learning model to improve the model's decentralized performance and tamper-proof properties, which provides a new perspective on the design of secure FL aggregation mechanisms. Moreover, these interactions can be costly in FL training because clients are geographically distributed, have limited computing and battery power, and experience varying network conditions. Using the BBGLR protocol multiple times to perform summations during model training is costly. Liu et al. [38] presented an efficient dropout-resilient aggregation for privacy-preserving machine learning. To achieve lightweight FL, Ma et al. [7] developed a Flamingo system designed to handle multi-round single-server aggregation, with a focus on privacy-preserving FL applications. Besides, Fotohi et al. [40] proposed a lightweight and secure deep learning model for privacy-preserving FL to improve traditional models' efficiency and privacy challenges. This system eliminates the need for a per-round setup, unlike previous protocols. It introduces a new lightweight dropout resilience that ensures that the server can still obtain a meaningful result if clients drop out in the middle of a sum. However, while the existing works address the issue of client dropout during privacy-preserving FL training, there is a lack of adequate verification for the correctness of the aggregation results.

Besides, Xu et al. [30] presented the VerifyNet system, which utilizes SS, a key agreement protocol, and a homomorphic hash function to provide security for each client. Ren et al. [41] have suggested improving privacy and verifiability in FL. This involves using linear homomorphic hash and digital signatures to verify the aggregation results and pinpoint the epoch when errors occurred. However, this method may lead to increased communication overhead. Luo et al. [27] used a basic signature method for the verification problem, where each client only needs to verify an aggregated signature. This signature is independent of the number of clients. However, the works mentioned above mainly focus on multi-round aggregation scenarios in FL, which achieve an efficient, single-server, secure aggregation scheme by addressing only one of the issues mentioned above. Existing schemes fail to meet comprehensive functional requirements, such as lightweight parameter reuse, client dropout-resilience, and aggregation result verification. Therefore, they cannot construct a truly efficient, single-server, secure aggregation architecture.

III. PRELIMINARY

We will introduce some basic knowledge that is used in our proposed framework. This will provide a clear description and definition of the framework.

A. Federated Learning

FL was first introduced in the literature [42] as a novel deep learning structure, which consists of two components: a set of clients $\mathbb{C}' = \{c_1, \ldots, c_j, \ldots, c_N\}$ and a server $\mathcal{S}$. They each handle their respective tasks to achieve collaborative computing between the client c_j and the server $\mathcal{S}$.

Local models training: The clients create local models by training their private datasets without sharing them, based on

Authorized licensed use limited to: Anhui University. Downloaded on October 13,2026 at 09:40:23 UTC from IEEE Xplore. Restrictions apply.

the initialization model from the server. When the clients receive the local models, the gradients from each round are sent to the server. For example, let's consider a scenario where the server $\mathcal{S}$ generates the initialization model G_0, and $\mathcal{N}$ clients will each generate a local model $\mathrm{M}_i = \{m_{(1,i)}, \ldots, m_{(j,i)}, \ldots, m_{(\mathcal{N},i)}\}$ in the i-th round. These local models are then sent to the server $\mathcal{S}$.

Server aggregation computing: The server $\mathcal{S}$ primarily handles the task of initialising the model and computing the aggregation of local models. When the server $\mathcal{S}$ receives the local models M_i in round i-th, the server $\mathcal{S}$ executes the local model aggregation computation. After the server $\mathcal{S}$ generates the global model G_i in the i-th round, G_i is sent to each client j, and the next round of local training is performed.

B. Pseudorandom Generators

The use of a pseudorandom generator (PRG) as a deterministic function means using a random seed in $\{0,1\}^{\lambda}$ to generate a longer string that is computationally indistinguishable from a truly random string. Note that λ is the computational security parameter. Besides, the PRG has two main characteristics: first, the length of the output is greater than that of the input, and second, the output result is a pseudorandom number. The whole process can be divided into the following three parts.

Seed Generation: The PRG first needs a seed λ as input generated by system time, user input, or other random sources. The seed λ choice will affect the final generated random number sequence.

Initialisation Phase: The use of the seed for the completion of the initial state of the pseudorandom generator. In subsequent calculations, the initialisation phase will be used as an alternative to the seed to generate subsequent pseudorandom numbers.

Iterative Calculation: According to a specific algorithm, the following pseudorandom number is generated by calculating the current state. This calculation process typically involves mathematical operations, shift operations, logical operations, and the use of a randomness enhancement algorithm to increase the randomness of the output sequence.

C. Shamir's Secret Sharing

Shamir's secret sharing (SS) is a method that divides a secret into multiple parts, called shares, and only a sufficient number of shares are needed to recover the original secret. This plays a significant role in cryptography applications and involves two steps.

Create secret shares: In Shamir's $t-out-of-\mathcal{N}$ secret sharing, the secret s is divided into $\mathcal{N}$ shares, which are sent to $\mathcal{N}$ users. The threshold t is required to reconstruct the secret $(t \leq \mathcal{N})$. The whole process can be shown as follows: $\mathbf{S.share}(s, t, \mathcal{N}) \rightarrow \{i, s_i\}_{i\in[1,\mathcal{N}]}$.

Secret shares reconstruction: To aid in the reconstruction of a secret s, we create a strategy for polynomial secret s reconstruction with a threshold of t. If the number of input secret shares is u (where $u > t$), we can reconstruct the secret s using the formula $\mathbf{S.recon}\{(i, s_i)\}_{i\in[1,u]} \rightarrow s$.

D. Homomorphic Hash Functions

To enable verification of aggregated results, we introduce the homomorphic hash function (HF). Informally, assume there is a message $x_i \in \mathbb{Z}_q$, which can be presented as $HF : \mathbb{Z}_q \leftarrow \mathbb{G}_1 \times \mathbb{G}_2$, the $\mathbb{G}_1$ and $\mathbb{G}_2$ are multiplicative cyclic groups with same prime order q. Similarly, the cyclic groups $\mathbb{G}_1$ and $\mathbb{G}_2$ have generators g and h, respectively. Therefore, the formula can also be expressed as follows.

$$HF(x_i) = (A_i, B_i) = (g^{HF_{\delta,\rho}(x_i)}, h^{HF_{\delta,\rho}(x_i)}) \tag{1}$$

where both δ and ρ are randomly selected secret keys in the finite field $\mathbb{Z}_q$. Also, $HF_{\delta,\rho}()$ is a one-way homomorphic hash function with two attributes.

Additive attribute: Assume we have two messages represented by x_1 and x_2, which can be presented as $HF(x_1 + x_2) = (g^{HF_{\delta,\rho}(x_1)+HF_{\delta,\rho}(x_2)}, h^{HF_{\delta,\rho}(x_1)+HF_{\delta,\rho}(x_2)})$.

Multiplication attribute: Assume there are a constant α and a message x_1, the multiplication attribute can be indicated as $HF(\alpha x_1) = (g^{\alpha HF_{\delta,\rho}(x_i)}, h^{\alpha HF_{\delta,\rho}(x_i)})$.

E. Bilinear Pairing

As for a bilinear pairing, it can be expressed as a map $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$. Informally, the bilinear pairing e has three properties.

Bilinearity: If there are two random numbers $a \in \mathbb{Z}_q^*$ and $b \in \mathbb{Z}_q^*$, we can obtain $e(g^a, h^b) = e(g, h)^{ab}$.

Computability: As for any $g \in \mathbb{G}_1$ and $h \in \mathbb{G}_2$, we can compute $e(g, h)$.

Non-degeneracy: $e(g, h) \neq 1$.

F. Diffie-Hellman Key Agreement Protocol

The Diffie-Hellman key agreement protocol was used to establish a shared key between any two users. Given a group $(\mathbb{G})$ with a prime order q, the public and secret keys for each user are generated as $\mathbf{KA.gen}(\mathbb{G}, g, q) \rightarrow (SK_j, g^{SK_j})$, where g is the generator of the group $\mathbb{G}$. Here, SK_j is the secret key, and g^{SK_j} is the public key. When client j knows the public key g^{SK_i} of client i, the shared key between users j and i is computed as $\mathbf{KA.agree}(SK_j, g^{SK_i}) \rightarrow s_{j,i}$. In practice, this shared key $s_{j,i}$ is often set to $HF((g^{SK_i})^{SK_j})$ for convenience.

IV. Problem Statement

In this section, we present the system model, threat model, and design goals to illustrate the proposed strategies clearly.

A. System Model

To address lightweight parameter settings, client dropout resilience, and aggregation result verification for FL, the proposed framework involves three parties: a trusted authority (TA), an aggregation server (AS), and a local client (LC), as illustrated in Fig. 2.

- *TA* is an essential element of our framework, fulfilling a significant function in creating public and private parameters offline. Subsequently, these parameters are sent securely to the local clients and server via a secure channel.

Authorized licensed use limited to: Anhui University. Downloaded on October 13,2026 at 09:40:23 UTC from IEEE Xplore. Restrictions apply.

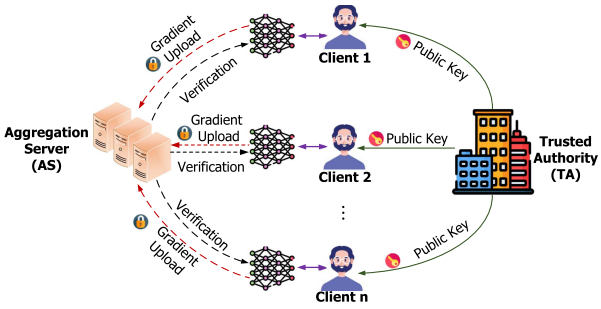


Fig. 2. System framework.

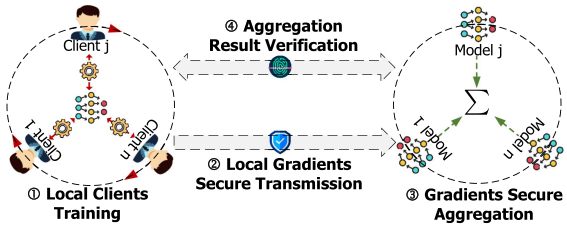


Fig. 3. System overview.

- *AS* is crucial to aggregating the local gradients uploaded by all online clients. Additionally, the system can generate a verification message for the aggregate results using the public/secret keys obtained from the TA.
- *LC*, as a training model entity, transmits the encrypted local gradients to the AS throughout each round. Besides, the LC and AS must engage in interactive computations and validate the aggregate result provided by the server, relying on the verification message.

B. Threat Mode and Design Goals

In this part, we define the threat model in our method. Specifically, the TA is an honest entity and will not collude with other entities in the system. The AS is an honest-but-curious (HBC) entity that can normally compute local gradients. However, we allow the AS to collude with multiple clients to gain more offensive capabilities. Besides, the AS can be controlled by a malicious adversary. To implement the proposed approach in our method and establish the threat model, we must achieve the following objectives.

- *Gradients Security:* Because the client owns the model, it contains critical information about the client's privacy. The value of the gradient indicates that the client model information is being transmitted. Therefore, the client's emphasis on privacy and security is critical to the system's overall stability.
- *Lightweight Training:* FL necessitates iterative calculations between many clients and the server as distributed training. Nevertheless, the training performance of the aggregation model is consistently affected by the security settings due to privacy protection regulations. Hence, the system must develop a single mechanism for setting safety parameters to meet the requirements for lightweight model training.
- *Security Verification:* Given the presence of collusion between the server and the client, and even if a malicious attacker controls the server, the system must have a mechanism for verifying the accuracy of each round of aggregation results.

V. Technical Details

In this section, we present a novel framework that innovatively addresses client dropout, efficiency, and verification for the multi-round federated training process. Initially, we guarantee the confidentiality of the local gradients provided by each client. Furthermore, since a significant number of our clients are involved in training, we must offer support for lightweight training. Specifically, it is essential to address inadequate data gathering from unstable networks and sudden client disconnections. Additionally, our proposed approach should enable clients to verify the correctness of the aggregate server, thereby preventing message tampering.

A. System Overview

As depicted in Fig. 3, the proposed framework comprises four stages: client training, secure transmission of local gradients, secure aggregation of local gradients, and verification of the aggregated results. A brief overview of the framework is provided below:

- *Client Training:* Our framework relies on numerous clients equipped with high-performance computation and data storage capabilities to perform training tasks using their own datasets. These clients consistently send the trained local gradients to a single aggregation server, which then generates the predictive aggregation model. Indeed, local clients are required to obtain the aggregation model to proceed with the following training phase, utilizing their own datasets. To avoid unreliable aggregation results from the server, each client must validate the correctness of the total aggregation obtained in each round. Hence, the training step comprises two primary components: iteratively updating the gradient and verifying the correctness of the aggregation gradients.
- *Local Gradients Secure Transmission:* After generating the local gradients, each client must transmit them to the server. To protect the confidentiality of local gradients, the clients use double-masking protocols to preserve the feature messages of the local gradients. For example, suppose there is a secret message x, and a random number is masked by adding r to x. Therefore, a client i generates local gradients with $\mathcal{N}$ dimensions, such as $\mathrm{M}_i = \{m_{(1,i)}, \ldots, m_{(j,i)}, \ldots, m_{(\mathcal{N},i)}\}$. Similarly, our framework can generate the double-masked vector $\mathcal{R}$ based on the dimension of $\mathcal{N}$. This vector is then used to compute $\mathrm{M}_i + \mathcal{R}$ to protect the privacy of local client information. After double-masking, the local gradient information can prevent information leakage in the transmitters and deter single-server mining of privacy information based on the received gradient values.

Authorized licensed use limited to: Anhui University. Downloaded on October 13,2026 at 09:40:23 UTC from IEEE Xplore. Restrictions apply.

- *Local Gradients Secure Aggregation:* When receiving the local gradients from the clients, the single server is responsible for aggregating them. Notably, the single server utilises the traditional aggregation (e.g., FedAvg) algorithm to execute operations on the local gradients it receives. Nevertheless, the server must tackle the issue of aggregating multiple local gradients. It still faces an issue where the clients cannot submit the gradient values in time, leading to clients dropping out.
- *Aggregation Result Verification:* It is essential to verify the aggregation results to ensure the resilience of the predictive model and take further measures to prevent problems, such as malicious attacks or collusion between the server and clients, as these can lead to misclassification errors in the aggregation model.

B. Secure Gradients Processing

According to the given information, the client has two tasks that need to be completed. First, they need to generate local gradients using their datasets. Second, they need to verify the accuracy of the aggregation results returned by the single server. Assume that all C' ($|\mathrm{C}'| = \mathcal{N}$) clients participate in training, which will generate the $\mathcal{N}$ local models $\mathrm{M} = \{\mathrm{M}_1, \mathrm{M}_2, \ldots, \mathrm{M}_{\mathcal{N}}\}$, where each local model M_i contains a series of gradient values, such as $\mathrm{M}_i = \{x_{1,i}, \ldots, x_{j,i}, \ldots\}$. To protect the privacy of each client's gradients, we use a double-masking approach to construct the privacy-preserving mechanism of our framework. In addition, client j and i each create a secret a_j and a_i which are then used to calculate g^{a_j} and g^{a_i}, respectively. Here, g^{a_j} represents client j's public key and g^{a_i} represents client i's public key. As a result, client j can secure the gradient value with the assistance of client i as follows.

$$\overrightarrow{\mathbf{Ma}}_j = \overrightarrow{\mathrm{M}}_j + \sum_{i \in \mathrm{M}(j), i<j} \mathbf{PRG}(r_{j,i}) - \sum_{i \in \mathrm{M}(j), i>j} \mathbf{PRG}(r_{j,i}) \tag{2}$$

where $r_{j,i} = g^{a_j a_i}$, which can be computed by client j since it has the secret a_j and client i's public key g^{a_i}. Note that these use Diffie-Hellman key exchange between client j and i. Additionally, we consider the output of the $\mathbf{PRG}$ as a vector of integers rather than a binary string. The term $\sum_j \pm \mathbf{PRG}(r_{j,i})$ is denoted as the double-making term for simplicity. Therefore, the client uses the double-making vector $\overrightarrow{\mathbf{Ma}}_j$ to protect their gradients without impacting the server's gradient aggregation.

However, the training of FL faces two main challenges in reality: (1) due to network instability, honest clients may not be able to submit training gradients promptly, and (2) clients may act maliciously or be forced to exit after being compromised by adversaries. Once the server times out, it will mark the connected clients as online and the rest offline, and simultaneously combine the local gradients from online sources. Owing to the presence of masks, gradient aggregation based on incomplete clients cannot fully eliminate the impact of mask values on the final aggregation result. To ensure the correct aggregation result, address the issue of offline client data loss, and resolve the need for parameter reset during multiple rounds of FL, we present a novel approach to constructing the graph with a communication cost that is independent of γ. Specifically, the graph is generated by a random string $v \in \{0,1\}^{\lambda}$ known to all participants in each round t. To enhance aggregation efficiency and avoid the need to reset the relevant parameters for each round, we utilise a selection strategy outlined in [7], called $ChooseSet(v, t, n_t, \mathrm{C}')$, which is used to identify the set of clients participating in round t. In round t, the set $S_t \subseteq \mathrm{C}'$ consists of n_t clients. Further, the server computes the aggregation model G_t as the graph among clients in S_t, denoted as $GenGraph(v, t, S_t) \rightarrow V_t$. A client $j \in S_t$ can use $FindNeighbors(v, S_t, j)$ to find its neighbors in V_t without materialising the entire graph, allowing for local generation of the neighbors of j. To ensure that the graph is connected with high probability in each round, we also select an appropriate ϵ value. This eliminates the need for setup in each round and enables multi-round aggregation. So, when the gradients from local clients are selected, the server uses gradient aggregation to obtain the aggregated model G_t in round t and sends it to each local client.

Based on the above mechanism, we utilise the SS protocol based on $t-out-of-\mathcal{N}$. Importantly, to avoid exposing the mask during the gradient reconstruction aggregation process, we introduce a new random variable e_j as an individual mask to design a double-masking mechanism, as shown in formula (3). So, the server needs to remove the mask $\mathbf{PRG}(r_{j,i})$ and $\mathbf{PRG}(e_j)$ to recover the correct sum. It requests each online client j to share the individual mask e_j with j's neighbors. Similarly, the offline client i is requested to share the secret a_i with i's neighbors. If the server receives shares from more than t neighbors, it can remove the masks and obtain aggregation results. Therefore, these shares enable the server to reconstruct either the double-masking or the individual mask.

$$\overrightarrow{\mathbf{Ma}}'_j = \overrightarrow{\mathrm{M}}_j + \sum_{i \in \mathrm{M}(j), i<j} \mathbf{PRG}(r_{j,i}) - \sum_{i \in \mathrm{M}(j), i>j} \mathbf{PRG}(r_{j,i}) + \mathbf{PRG}(e_j) \tag{3}$$

C. Verifiable Aggregation Model

Once the server obtains the masked local gradients, it can reconstruct and distribute the aggregation result to each client. Due to the potentially malicious nature of the server, it can provide inaccurate feedback regarding the aggregation results to the client. Then, we develop a verifiable approach highly compatible with the aforementioned double-masking protocol. This solution enables each client to quickly verify the correctness of the aggregation result provided by the server. To address this issue, we utilise a homomorphic hash function in combination with pseudorandom technology as the fundamental framework of our verifiable approach. Clients can utilise this capability to verify the correctness of operations performed by the server with minimal computational overhead. More importantly, the entire procedure can be categorised into three distinct stages: key generation, client execution, and server computation for verification.

- *Key Generator Step:* In the first step, we need to generate public and secret keys for each client j. The keys include (δ, ρ) and $K = (K_1, K_2)$, which are the secret keys used in homomorphic hash and pseudorandom functions, as well

Authorized licensed use limited to: Anhui University. Downloaded on October 13,2026 at 09:40:23 UTC from IEEE Xplore. Restrictions apply.

as (N_j^{PK}, N_j^{SK}) and (P_j^{PK}, P_j^{SK}). Additionally, the keys (N_j^{PK}, N_j^{SK}) and (P_j^{PK}, P_j^{SK}) will be shared to encrypt the client j's local gradients set M_j.

- *Clients Execution Step:* To verify the correctness of the x_j returned from the server, the client j needs to perform a series of hash computations of x_j as formula (4).

$$HF(x_j) = (A_j, B_j) = (g^{HF_{\delta,\rho}(x_j)}, h^{HF_{\delta,\rho}(x_j)}) \quad (4)$$

After the client j executes $PK_{K_1}(j) = (\gamma_j, v_j)$ and $PK_{K_2}(\tau) = (\gamma, v)$, it then computes the following formula (5).

$$PK_K(j, \tau) = (E_j, F_j) = (g^{\gamma_j\gamma + v_j v}, h^{\gamma_j\gamma + v_j v}) \quad (5)$$

Further, the client j calculates the values of L_j and Q_j by following the formulas (6) and (7).

$$L_j = (E_j \cdot A_j^{-1})^{\frac{1}{d}} = (g^{\gamma_j\gamma + v_j v - HF_{\delta,\rho}(x_j)})^{\frac{1}{d}} \quad (6)$$

$$Q_j = (F_j \cdot B_j^{-1})^{\frac{1}{d}} = (h^{\gamma_j\gamma + v_j v - HF_{\delta,\rho}(x_j)})^{\frac{1}{d}} \quad (7)$$

where d is a selected positive integer.
Finally, the client j will send the computing set $\rho_j = (\hat{x}_j, A_j, B_j, L_j, Q_j, \Omega_j = 1)$.

- *Server Computation Step:* To complete the entire verification process, the server also needs to generate the relevant Proof of verification. Specifically, the server provides each client with the results along with $Proof(A, B, L, Q, \Omega)$ in the following formulas.

$$A = \prod_{j=1}^{j=|\mathrm{M}_j|} A_j; \quad B = \prod_{j=1}^{j=|\mathrm{M}_j|} B_j \quad (8)$$

$$L = \prod_{j=1}^{j=|\mathrm{M}_j|} L_j; \quad Q_j = \prod_{j=1}^{j=|\mathrm{M}_j|} Q_j; \quad \Omega = \prod_{j=1}^{j=|\mathrm{M}_j|} \Omega_j \quad (9)$$

The server will broadcast the computing result $P_r = \{\rho = \sum x_j, A, B, L, Q, \Omega\}$ to each client.

Finally, according to the $Proof$ result, each client decides whether to accept or reject the aggregation result and proceeds with the next iteration. As for each training round, each client knows $PK_{K_1}(j) = (\gamma_j, v_j)$ and $PF_{K_2}(\tau) = (\gamma, v)$, and further computes $\Psi = \sum_{j\in N}(\gamma_j\gamma + v_j v)$ and $\Phi = e(g, h)^{\psi}$. We need to determine if the following equalities hold as in the following formula (10).

$$(A, B) \stackrel{?}{=} (A', B'), e(A, h) \stackrel{?}{=} e(g, B);$$
$$e(L, h) \stackrel{?}{=} e(g, Q), \Phi \stackrel{?}{=} e(A, h) \cdot e(L, h)^d \quad (10)$$

According to the above equation, if any equation is not satisfied, we should reject the aggregate result; if they are all valid, then we can accept the result and continue.

In brief, we provide a concise explanation of the proposed framework by outlining the algorithm of the entire verification approach, which is constructed within a privacy-preserving multi-round FL algorithm to satisfy the functions of clients' dropout-resilience, lightweight, and verification, namely Algorithm 1 and Algorithm 2.

Algorithm 1: Lightweight Privacy-Preserving Multi-Round FL Algorithm.

Input: Clients Gradients $\mathrm{M} = \{x_1, \ldots, x_j, \ldots, \}$;
Output: Aggregation Model G_t in t round;
Initialization: Generate keys (N_j^{PK}, N_j^{SK}) and (P_j^{PK}, P_j^{SK}); using homomorphic hash and pseudorandom functions to get $K = (K_1, K_2)$ and (δ, ρ);

Clients:
Send the public keys (N_j^{PK}, P_j^{PK}) to the server through a secure channel.

Server:
Broadcast $\{m, N_m^{PK}, P_m^{PK}, \tau = sum\}_{m\in\mathrm{M}_t}$, where m is the client and $\tau = sum$ represents the statistic label to be calculated.

Clients:
- Receive the $\{m, N_m^{PK}, P_m^{PK}, \tau = sum\}_{m\in\mathrm{M}_t}$ from the server. Check whether the $|\mathrm{M}_t| \le t$ and all of the key pairs (N_m^{PK}, P_m^{PK}) are distinctive. If not, abort and start over.
- Select a random number β_j to obtain the shares, such as $\{(m, \beta_{j,m})\}_{m\in\mathrm{M}_t} \leftarrow \mathbf{S.share}(\beta_j, t, \mathrm{M}_t)$, where $\beta_{j,m}$ is the share of client j to client m;
- Generate the shares of N_j^{PK} as $\{(m, N_{j,m}^{SK})\}_{m\in\mathrm{M}_t} \leftarrow \mathbf{S.share}(N_j^{SK}, t, \mathrm{M}_t)$, where $N_{j,m}^{SK}$ is the share of client j to client m;
- Compute $\mathcal{R}_{j,m} \leftarrow \mathbf{AE.enc}(\mathbf{KA.agree}(P_j^{SK}, P_m^{PK}), j||m||N_{j,m}^{SK}||\beta_{n,m})_{m\in\mathrm{M}_t}$, where $\mathbf{AE.enc}()$ denotes a symmetric encryption $\mathbf{KA.agree}(P_j^{SK}, P_m^{PK})$;
- Send $\{\mathcal{R}_{j,m}\}_{m\in\mathrm{M}_t}$ to the server.

Server:
Receive information from at least t clients. Broadcast $\{\mathcal{R}_{j,m}\}$ to each client.

Clients:
- Receive $\{\mathcal{R}_{j,m}\}$ and compute the shared key with the client m, such as $s_{j,m} \leftarrow \mathbf{KA.agree}(N_j^{SK}, N_m^{PK})$;
- Encrypt the local gradients as $\hat{x}_j = x_j + \mathbf{PRG}(\beta_j) + \sum_{m\in\mathrm{M}_t; j<m} \mathbf{PRG}(s_{j,m}) - \sum_{m\in\mathrm{M}_t; j>m} \mathbf{PRG}(s_{m,j})$.

Return $\sum x_j$.

VI. Security Analysis

To illustrate the security of our framework, we will describe the privacy-preserving and verification aspects. Additionally, we will analyse how the framework ensures the confidentiality of each client's gradients.

A. Privacy-Preserving Analysis

Our framework guarantees the confidentiality and integrity of data and models in an HBC setting. According to our defined threat model, the server can collude with any $t-1$ clients to get the most offensive capabilities. Nevertheless, they lack knowledge about honest clients' local gradients, except for the aggregation results. Furthermore, the server is under the control

Authorized licensed use limited to: Anhui University. Downloaded on October 13,2026 at 09:40:23 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: Verification Aggregation Results Returned Algorithm.

Input: Clients Gradients x_j, δ, ρ;
Output: Verification Returns True or False;
Clients:
- Compute $HF(x_j) = (A_j, B_j) = (g^{HF_{\delta,\rho}}, h^{HF_{\delta,\rho}})$;
- Compute $PK_{K_1}(j) = (\gamma_j, v_j)$; $PF_{K_2}(\tau) = (\gamma, v)$;
- Compute $PF_K(j, \tau) = (g^{\gamma_j\gamma + v_j v}, h^{\gamma_j\gamma + v_j v})$;
- Calculate $L_j = (g^{\gamma_n\gamma + v_n v - HF_{\delta,\rho}(x_j)})^{\frac{1}{d}}$;
- Calculate $Q_j = (h^{\gamma_n\gamma + v_n v - HF_{\delta,\rho}(x_j)})^{\frac{1}{d}}$;
- Send $\rho_j = \{\hat{x}_j, A_j, B_j, L_j, Q_j, \Omega_j = 1\}$ to the server.

Server: Calculate the $Proof_v\{A, B, L, Q, \Omega\}$ as:
- Compute $A = \prod_{j=1}^{j=|\mathrm{M}_j|} A_j$ and $B = \prod_{j=1}^{j=|\mathrm{M}_j|} B_j$;
- Compute $L = \prod_{j=1}^{j=|\mathrm{M}_j|} L_j$ and $Q = \prod_{j=1}^{j=|\mathrm{M}_j|} Q_j$;
- Compute $\Omega = \prod_{j=1}^{j=|\Omega_n|}$;
- Broadcast $P_r = \{\rho = \sum x_j, A, B, L, Q, \Omega\}$.

Clients:
Verify $(A, B) \stackrel{?}{=} (A', B')$, $e(A, h) \stackrel{?}{=} e(g, B)$, $e(L, h) \stackrel{?}{=} e(g, Q)$, $\Phi \stackrel{?}{=} e(A, h) \cdot e(L, h)^d$.
Return True or False.

of a malicious attacker and cannot access private data belonging to honest clients or the model's confidential information.

Specifically, each client's gradient x_j can be encrypted using formula (3). The single-masking is created in pairs using the Diffie-Hellman protocol to prevent the server from deriving the client's original gradients. However, the client can drop out during the federated training process. To achieve lightweight training, each graph V_t remains connected even after removing the vertices corresponding to dropped clients [7]. Furthermore, we utilise the SS strategy to enable clients to upload local models to the server smoothly. In the $t{-}out{-}of{-}\mathcal{N}$ setting of the SS strategy, the server can reconstruct the local gradients of $\mathcal{N}$ clients as long as the number of clients submitted for each round of federated training is not less than t, which can be represented as $\mathbf{S.share}(s, t, \mathcal{N}) \rightarrow \{i, s_i\}_{i\in[1,\mathcal{N}]}$. After applying the SS protocol, we employ double-masking to prevent privacy leakage resulting from potential collusion between the server and clients. To clarify, our secure aggregation protocol is primarily designed to ensure the privacy and integrity of each client's contribution during the aggregation phase. This guarantee remains in effect even in the presence of an HBC server and an HBC majority of clients. Our scheme explicitly prevents the server from learning individual client updates. Even if the server colludes with a subset of clients, the privacy of client models remains protected as long as the number of colluding clients does not exceed a threshold t, which is determined by the underlying cryptographic primitive.

Furthermore, we utilize the double-masking technique, offering an efficient privacy of gradients for FL, which primarily relies on simple additive operations, resulting in significantly reduced computational and communication overhead. HE provides strong privacy guarantees by enabling direct computation on encrypted data. However, HE schemes generally entail substantial computational and communication costs due to data expansion and the complexity of cryptographic operations. Such overheads often limit their practical use to simpler model architectures or require considerable hardware resources. We focus on secure aggregation in scenarios involving a large number of participating clients, where there are stringent demands on both the efficiency of the secure aggregation protocol and its robustness to client dropouts.

B. Verification Analysis

Upon receiving the collection $\{\rho, A, B, L, Q, \Omega\}$ from the server, the clients verify the equation $\Phi = e(A, h) \cdot e(L, h)^d$. The equation is valid only if the total of $\gamma_n\gamma$ and $v_n v$ overall j in set M_j is included in L. If this requirement is satisfied, clients may infer the outcome using the formula (11), which implies that $A = g^{HF_{\delta,\rho}(\sigma)}$.

$$L = \prod_{j=1}^{|\mathrm{M}_j|} \left(g^{\gamma_n\gamma + v_n v - HF_{\delta,\rho}(x_j)}\right)^{\frac{1}{d}} \tag{11}$$

Subsequently, clients should confirm the correctness of both $e(A, h) = e(g, B)$ and $e(L, h) = e(g, Q)$. When these requirements are fulfilled, clients can get confidence that the server has accurately calculated B and Q. Similarly, clients need to verify the accuracy of the calculation of (A, B). Finally, if $HF(\sigma) = (A', B') = (A, B)$ is true, each client can be sure that the server has provided the correct aggregation result $\sigma = \sum x_j$.

Besides, compared to traditional FL verification techniques, the advantage of our verification method is achieved through: (1) Optimised Client-side Verification: each client performs a fixed, minimal number of pairing operations for verification, regardless of the total number of clients or the model size. This ensures lightweight participation for individual clients. (2) Efficient Server-side Aggregation: The server primarily aggregates client-provided proofs using efficient group multiplications, scaling linearly with the number of clients. Notably, the server avoids computationally expensive pairing operations during the main aggregation phase. (3) Distinct Design Philosophy: Our framework separates lightweight secure aggregation from the verification process, enabling a streamlined design where verification focuses on the integrity of the server's aggregated result, rather than requiring complex and resource-intensive proofs over individual client contributions. This approach provides strong integrity guarantees with controlled overhead.

As shown in Formula (11), the verification computations performed by each client mainly involve calculating L, which involves computing the hash function HF for each client, followed by exponentiation and multiplication operations. These operations have low computational complexity for a single client and are independent of the total number of clients. Subsequently, clients need to verify $e(A, h) = e(g, B)$ and $e(L, h) = e(g, Q)$. These are pairing-based verification operations, whose computational complexity is typically related to the base of the bilinear pairing, not the number of clients. Each verification only requires a fixed number of bilinear pairing computations, which is of constant time complexity. Finally, verify $HF(\sigma)$, which

Authorized licensed use limited to: Anhui University. Downloaded on October 13,2026 at 09:40:23 UTC from IEEE Xplore. Restrictions apply.

is also a hash function and equality check, incurring minimal computational overhead. Therefore, the verification method proposed in this paper is capable of handling participation scenarios involving thousands of clients.

VII. Performance Evaluation

In this section, we will evaluate the performance of our framework in terms of its functionality, including classification accuracy, verification accuracy, and client dropout resilience. The experiments are performed on a personal computer operating on 64-bit Windows 11, equipped with a 2.60 GHz 13th Generation Intel(R) Core(TM) i5-13500H CPU and 32 GB of RAM. We assessed the performance of the method on 1000 clients using the MNIST, CIFAR-10, and FEMNIST datasets under an IID setup. To better balance security and performance, we employ an Elliptic-Curve-based key agreement protocol for key distribution between two clients, and use standard Shamir's $t-out-of-\mathcal{N}$ secret sharing to generate secret shares [43]. Additionally, we utilise AES in counter mode and AES-GCM with 128-bit keys for authenticated encryption and as a pseudorandom generator, respectively. To ensure the stability of our experiments, the results presented in the paper are calculated as the average of five experiments performed.

A. Functionality

As shown in Table I, we evaluate the functionality of our primary works by comparing them to current studies such as [28] and [7], [29], [30]. Table I indicates that the research primarily emphasises privacy-preserving throughout the training execution process [28]. Nevertheless, these efforts fail to tackle the problem of client dropout in FL. Besides, the literature [7], [29], [30] addressed the issue of client dropout during federated training. However, it lacks support for efficient public/secret key reuse and validation of aggregation results. In contrast to the works above, our framework enables clients to verify the aggregation results generated by the server while ensuring the confidentiality of clients' local gradients and managing the client's dropout using lightweight parameter reuse throughout the entire federated training process.

B. Classification Accuracy

In this section, we use the MNIST dataset to evaluate the classification accuracy of the proposed framework. The experiments are carried out on a Multilayer Perceptron (MLP) network with three layers: input layer, hidden layer, and output (e.g., input layer = 28x28, hidden layer = 200, hidden layer = 200, output = 10, learning rate = 0.01, local batch size = 10). To demonstrate the performance of the proposed framework in this paper, we assess its performance in FL based on two dimensions: the number of clients involved in the training process and the magnitude of the local gradients possessed by these clients. Generally, the accuracy of an aggregation model is often correlated with the number of gradients and clients implemented during training, as well as the system's computational cost and communication overhead. Nevertheless, increasing the

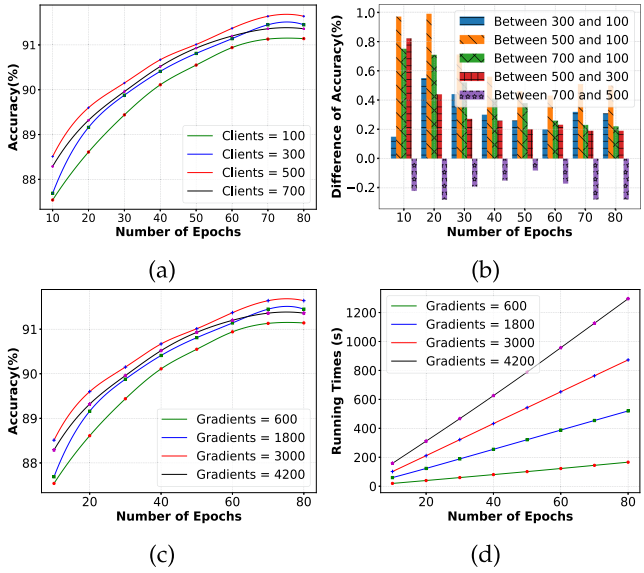


Fig. 4. $|C| = 100$, (a) No dropout, classification accuracy with varying numbers of clients; (b) No dropout, the difference in classification accuracy varies with the number of clients; (c) No dropout, classification accuracy with a different number of gradients; (d) No dropout, run time with a different number of gradients.

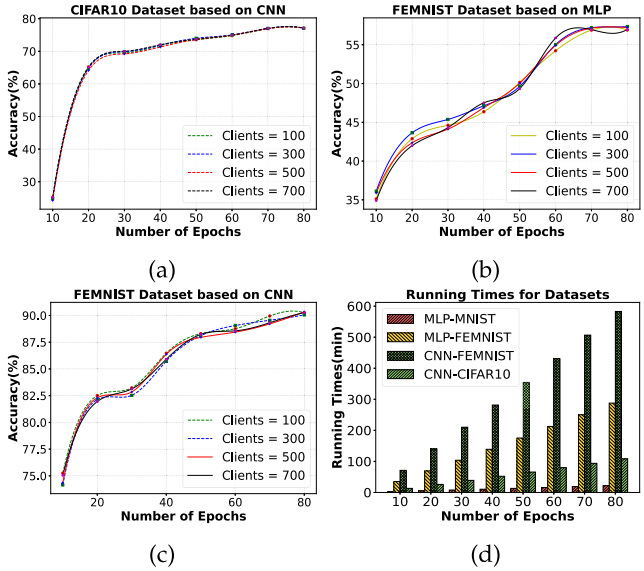


Fig. 5. $|C| = 100$, (a) No dropout, training the CIFAR-10 dataset using CNN; (b) No dropout, training the FEMNIST dataset using MLP; (c) No dropout, training the FEMNIST dataset using CNN; (d) No dropout, running times for different datasets.

quantity of training rounds does not always result in boundless enhancements in accuracy. To further analyze classification accuracy using different metrics, we compare the accuracy across various numbers of participating clients and provide the runtime for different gradient computations, as shown in Fig. 4.

According to Fig. 4(a), we observe that the classification accuracy improves with the increasing number of training rounds, even with no clients dropping out. However, after approximately 70 iterations, the accuracy stabilizes and shows no significant

Authorized licensed use limited to: Anhui University. Downloaded on October 13,2026 at 09:40:23 UTC from IEEE Xplore. Restrictions apply.

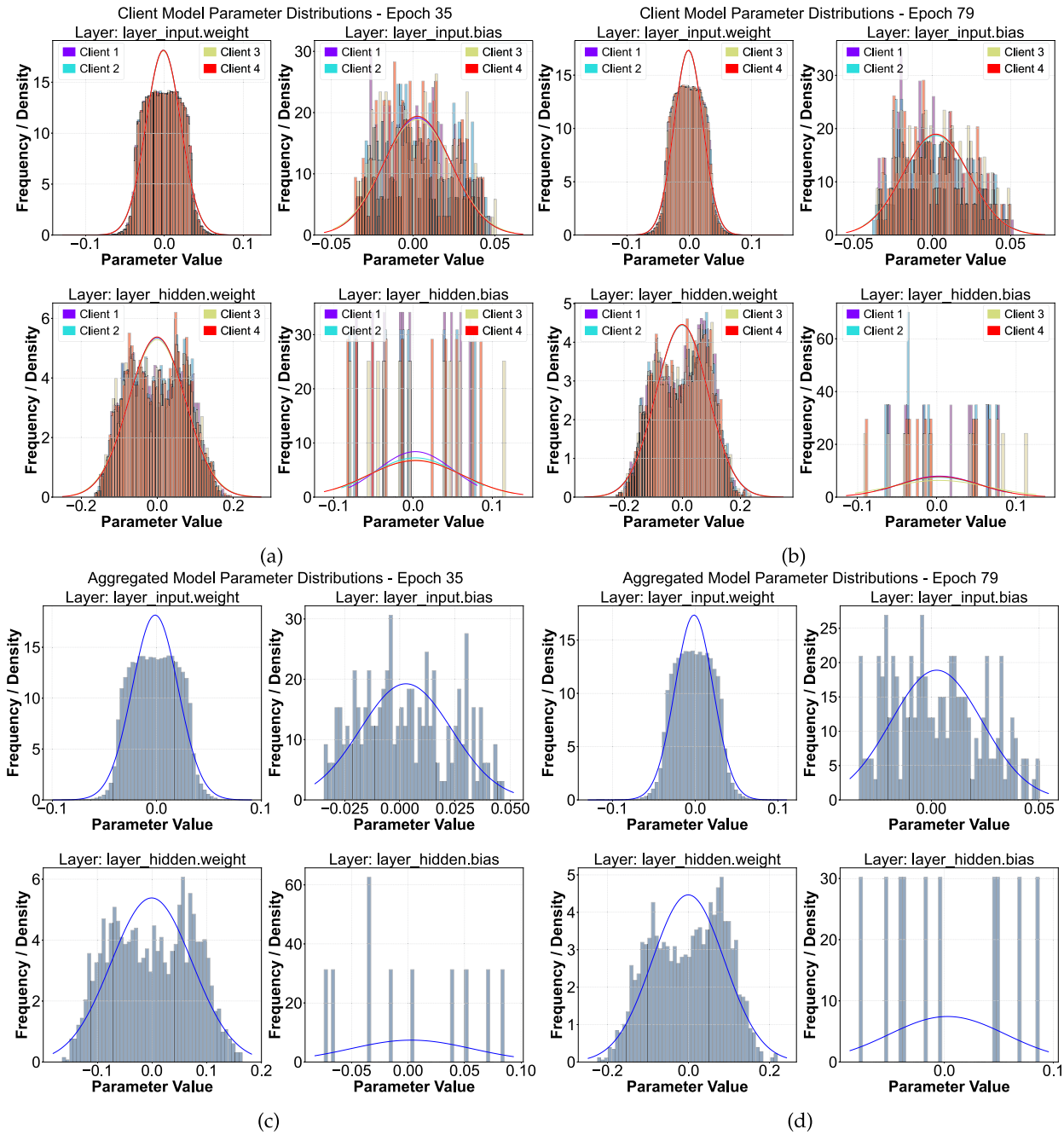


Fig. 6. (a) No dropout, distribution of some local model parameters at the epoch = 35; (b) No dropout, distribution of local model parameters at the epoch = 79; (c) No dropout, distribution of aggregated model parameters at the epoch = 35; (d) No dropout, distribution of aggregated model parameters at the epoch = 79.

changes. Generally, as the number of clients increases, the rate of improvement in classification accuracy tends to decrease. The vertical axis in Fig. 4(b) represents the variations in classification accuracy among different clients, while the horizontal axis illustrates the number of distinct clients. From Fig. 4(b), we can see that the classification accuracy decreases when the number of clients increases from 500 to 700, as indicated by the negative difference in the last segment. To further illustrate the complexity of the system proposed in this paper, we conducted a statistical analysis of the total gradient during training while achieving the same classification accuracy demonstrated in Fig. 4(c). It is clear from Fig. 4(c) that there is a consistent relationship between the total gradient computation and the change in the number of clients. Fig. 4(d) shows that as the number of computed gradients increases, the time required to train the entire system increases rapidly.

To further validate the effectiveness of the scheme proposed in the paper, we also evaluated the performance of our method with 1000 clients using the CIFAR-10 and FEMNIST datasets. Specifically, we employed convolutional neural networks (CNNs) with three layers (input, hidden, and output) to implement secure federated aggregation on the CIFAR-10

Authorized licensed use limited to: Anhui University. Downloaded on October 13,2026 at 09:40:23 UTC from IEEE Xplore. Restrictions apply.

and FEMNIST datasets, demonstrating the effectiveness of our method on complex real-world datasets, as shown in Fig. 5. From Fig. 5(a) to (c), it is evident that when training different datasets and utilizing various neural networks, the classification accuracy of the models increases with the number of federated training rounds until it eventually plateaus. However, as shown in Fig. 5(d), the training time for different datasets varies, primarily due to the significant differences in dataset sizes. Specifically, training on larger datasets, such as FEMNIST, requires more time and resources. Therefore, the method proposed in this paper is consistent with the training time required for traditional deep learning approaches and proves effective on complex datasets.

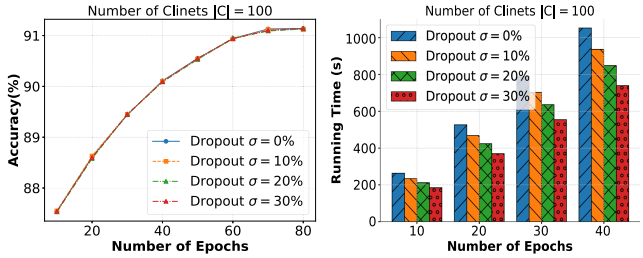


Fig. 7. (a) Comparison classification accuracy between no dropout and dropout; (b) Comparison running time between no dropout and dropout.

C. Verification Accuracy

Our system model assists each client in verifying the accuracy of aggregation results provided by the server. To perform this verification, the server must supply a $Proof$ to each client to attest to the correctness of the aggregation results. This Proof serves as the certificate for accepting or rejecting the results. To demonstrate the accuracy of the verification method, we randomly selected local models and aggregated them based on those generated from four clients at different epochs. This was done under the condition of a total training epoch count of 80. We then plotted the distribution of the model parameters. Additionally, the server needs to calculate the combined models and their corresponding $Proof$ for each client. If the verification is successful, the distribution of local model parameters used to create the combined model parameters in the same epoch should be similar, as shown in Fig. 6.

Fig. 6 illustrates the distributions of local model parameters at two different epochs. The parameter distributions for their corresponding aggregation models are shown in Fig. 6(a) and (c), or Fig. 6(b) and (d). As shown in Fig. 6, we can observe that the distribution of parameters for each layer of clients' models or aggregation models follows a normal distribution. After the server successfully executes the aggregation for a significant number of local models, the parameters of the aggregation model remain unchanged. This is confirmed by verifying the accuracy of the result obtained from the server using a $Proof$.

D. Processing of Clients Dropout

The entire framework is designed to be robust enough to handle situations where participating clients drop out during training due to device battery and hardware limitations. Additionally, there are occasions when the network suddenly disconnects while clients are transmitting local gradients. To evaluate how the system performs when clients drop out, we will simulate scenarios in which $10\%, 20\%$, and 30% of the clients leave, based on an initial pool of 100 clients. This simulation is depicted in Fig. 7.

As illustrated in Fig. 7(a), using a total client count of 100 as the baseline, the framework's classification accuracy remained consistent even when $10\%, 20\%$, and 30% of clients exited individually. However, regarding the time required for the system to handle varying numbers of client dropouts, Fig. 7(b) illustrates that as the number of existing clients increased, the time taken for system module execution decreased. This is primarily due to the varying solution times required for the SS protocol under different threshold t.

Besides, we set a threshold value t using SS, indicating that at least t client shares are required to reconstruct the secret. This threshold t also represents the maximum number of dropouts allowed among federated learning clients. If we set t to a low value ($t \ll \mathcal{N}$), the server can aggregate the model even if many clients drop out. This increases the system's robustness and resilience, reducing the risk of aggregation failure due to client dropout. However, a lower t means fewer client contributions are included in the secure aggregation. As a result, the aggregation may favour the data distribution of active or surviving clients. If their local data does not represent the global distribution, the model may become biased. Additionally, with fewer clients participating in each round, the model learns less information, potentially requiring more rounds of federation to achieve the same level of accuracy, which can impair the generalisation and accuracy of the final aggregated model. Therefore, we set up four different scenarios with client dropout rates of 0%, 10%, 20%, and 30%, ensuring the reliability and classification accuracy of the final aggregated model even when federated secure aggregation is performed under most conditions.

To demonstrate the practicality of the method outlined in this paper, we performed statistical analyses of the system model by testing each function. This involved assessing the communication head for executing each security module individually with a total of N' epochs for $\mathrm{C} = 100$ clients $(\mathrm{C}' = 1000)$ according to different drop rates for clients σ in Tables II and III. Table II indicates that client communication overhead is primarily attributed to three main components: masking agreement, masked input, and model verification. Based on Table II, it is evident that the primary communication load for clients during the secure training of local models primarily involves the transmission of model parameters. In contrast, the communication overhead required by various security protocols to ensure the safety of local models is significantly lower than that of the former. Regarding the server, the primary communication overhead stems from the size of the aggregated model. The additional communication overhead mainly arises from the computation process of the mask reconstruction aggregation model, as shown in Table III. Based on Tables II and III, we can conclude that the mechanism proposed in this paper satisfies the requirements

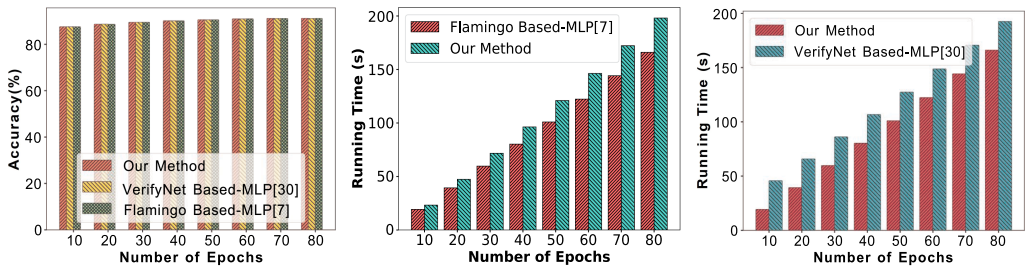
Authorized licensed use limited to: Anhui University. Downloaded on October 13,2026 at 09:40:23 UTC from IEEE Xplore. Restrictions apply.

TABLE II
COMMUNICATION OVERHEAD OF TOTAL EPOCHS N' FOR CLIENTS $\mathrm{C} = 100$

	Dropout	Masking Agreement	Unmasked Input	Masked Input	Model Verification	Total
Client	$\sigma = 0$	154KB	73MB$\times N'$	73MB$\times N'$ + 77KB	34.4KB$\times N' \times \mathrm{C} \times (1-\sigma)$	$\approx$ 73MB$\times N'$
Client	$\sigma = 10\%$	154KB	65.7MB$\times N'$	65.7MB$\times N'$ + 77KB	34.4KB$\times N' \times \mathrm{C} \times (1-\sigma)$	$\approx$ 65.7MB$\times N'$
Client	$\sigma = 20\%$	154KB	58.4MB $\times N'$	58.4MB $\times N'$ + 77KB	34.4KB$\times N' \times \mathrm{C} \times (1-\sigma)$	$\approx$ 58.4MB$\times N'$
Client	$\sigma = 30\%$	154KB	51.1MB$\times N'$	51.1MB$\times N'$ + 77KB	34.4KB$\times N' \times \mathrm{C} \times (1-\sigma)$	$\approx$ 51.1MB$\times N'$

TABLE III
COMMUNICATION OVERHEAD OF TOTAL EPOCHS N' FOR SERVER BASED ON $\mathrm{C} = 100$

	Dropout	Models Receive	Mask Reconstruction	Aggregated Model	Model Verification	Total
Server	$\sigma = 0$	73MB$\times N'$ + 77KB	241.7KB	73MB	0	73MB +318.7KB
Server	$\sigma = 10\%$	65.7MB$\times N'$ + 77KB	241.7KB	73MB	0	73MB + 318.7KB
Server	$\sigma = 20\%$	58.4MB $\times N'$ + 77KB	241.7KB	73MB	0	73MB + 318.7KB
Server	$\sigma = 30\%$	51.1MB$\times N'$ + 77KB	241.7KB	73MB	0	73MB + 318.7KB



(a) (b) (c)

Fig. 8. (a) No dropout, $\mathrm{C} = 100$, comparison classification accuracy between different existing method; (b) No dropout, $\mathrm{C} = 100$, comparison running time between Flamingo and our method; (c) No dropout, $\mathrm{C} = 100$, comparison running time between our method and VerifyNet.

for lightweight implementation to achieve secure multi-round federated training.

Additionally, we compared the accuracy and communication overhead between training the MPL network without any security modules and our proposed system model, as shown in Fig. 8. In Fig. 8, a comparison is shown between the algorithm proposed in this paper and existing methods when there are no client dropouts and the number of clients is $|\mathrm{C}| = 100$. In Fig. 8(a), three methods were used to conduct federated training for the MLP network. The classification accuracy improved as the number of training epochs increased, and the aggregated model's classification precision remained consistently high across all three methods in each epoch. To further demonstrate the effectiveness of the method proposed in this paper, Fig. 8(b) and (c) compare the running time of this method with two others. Fig. 8(b) shows that our method takes longer than the Flagingo method. However, the Flagingo method does not have a verification mechanism that returns aggregation results from the server. Fig. 8(b) also demonstrates that throughout the entire training process of the model, its runtime increases by approximately 30 seconds. This increase does not impose a significant computational burden on the model's training code. Similarly, our method requires less runtime during model training than VerifyNet in Fig. 8(c). The main reason for this phenomenon is the difference in the key-sharing mechanism used by both methods. To ensure the security of local client models, the VerifyNet method needs to perform this sharing process before each training round. In contrast, our method benefits from a graphical structure, which enables it to complete the model training with just one key-sharing execution.

VIII. CONCLUSION

This paper presents a framework for validating the aggregation results from the server, which is designed to handle unexpected client dropouts during federated training and achieve secure key reuse to support lightweight federated training. Furthermore, the security analysis demonstrates the high security level of our method under the honest-but-curious threat model. Finally, the experiments conducted on real-world datasets demonstrate the practical performance of our proposed framework. In the future, our research will focus on security priorities and highly flexible federated systems, and we will study federated training under non-independent and identically distributed data to enhance the versatility and stability of federated systems in complex environments.

REFERENCES

[1] M. Alazab, R. M. S. Priya, P. M., P. K. R. Maddikunta, T. R. Gadekallu, and Q. Pham, "Federated learning for cybersecurity: Concepts, challenges, and future directions," *IEEE Trans. Ind. Informat.*, vol. 18, no. 5, pp. 3501–3509, May 2022.

Authorized licensed use limited to: Anhui University. Downloaded on October 13,2026 at 09:40:23 UTC from IEEE Xplore. Restrictions apply.

[2] R. Zhu, M. Li, J. Yin, L. Sun, and H. Liu, "Enhanced federated learning for edge data security in intelligent transportation systems," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 11, pp. 13396–13408, Nov. 2023.
[3] S. H. Alsamhi et al., "Federated learning meets blockchain in decentralized data sharing: Healthcare use case," *IEEE Internet Things J.*, vol. 11, no. 11, pp. 19602–19615, Jun. 2024.
[4] J. Tang, "Global research and application trends report on federated learning 2023," Website, 2023. [Online]. Available: https://federated-learning.org/fl-ijcai-2023/
[5] A. R. Elkordy and A. S. Avestimehr, "HeteroSAg: Secure aggregation with heterogeneous quantization in federated learning," *IEEE Trans. Commun.*, vol. 70, no. 4, pp. 2372–2386, Apr. 2022.
[6] Z. Liu, H. Lin, and Y. Liu, "Long-term privacy-preserving aggregation with user-dynamics for federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 2398–2412, 2023.
[7] Y. Ma, J. Woods, S. Angel, A. Polychroniadou, and T. Rabin, "Flamingo: Multi-round single-server secure aggregation with applications to private federated learning," in *Proc. 44th IEEE Symp. Secur. Privacy*, San Francisco, CA, USA, 2023, pp. 477–496.
[8] A. Mondal, Y. More, R. H. Rooparaghunath, and D. Gupta, "Poster: FLATEE: Federated learning across trusted execution environments," in *Proc. IEEE Eur. Symp. Secur. Privacy*, Vienna, Austria, 2021, pp. 707–709.
[9] F. Tang, C. Wen, L. Luo, M. Zhao, and N. Kato, "Blockchain-based trusted traffic offloading in space-air-ground integrated networks (SAGIN): A federated reinforcement learning approach," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 12, pp. 3501–3516, Dec. 2022.
[10] S. Guo et al., "Sandbox computing: A data privacy trusted sharing paradigm via blockchain and federated learning," *IEEE Trans. Comput.*, vol. 72, no. 3, pp. 800–810, Mar. 2023.
[11] H. U. Sami and B. Güler, "Secure aggregation for clustered federated learning with passive adversaries," *IEEE Trans. Commun.*, vol. 72, no. 7, pp. 4117–4133, Jul. 2024.
[12] Y. Wu, M. Xue, D. Gu, Y. Zhang, and W. Liu, "Sample-specific backdoor based active intellectual property protection for deep neural networks," in *Proc. 4th IEEE Int. Conf. Artif. Intell. Circuits Syst.*, Incheon, Republic of Korea, 2022, pp. 316–319.
[13] Y. Li, M. Zhu, X. Yang, Y. Jiang, T. Wei, and S. Xia, "Black-box dataset ownership verification via backdoor watermarking," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 2318–2332, 2023.
[14] Q. Chen, P. Zheng, Y. Du, W. Luo, and H. Liu, "FedMCT: A federated framework for intellectual property protection and malicious client tracking," in *Proc. 16th Int. Conf. Mach. Learn. Comput.*, Shenzhen, China, 2024, pp. 80–86.
[15] D. Pasquini, D. Francati, and G. Ateniese, "Eluding secure aggregation in federated learning via model inconsistency," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Los Angeles, CA, USA, 2022, pp. 2429–2443.
[16] T. Eltaras, F. Sabry, W. Labda, K. Alzoubi, and Q. Ahmedeltaras, "Efficient verifiable protocol for privacy-preserving aggregation in federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 2977–2990, 2023.
[17] S. Kim, H. Yun, M. LeeGubri, S. Yoon, and S. J. Oh, "ProPILE: Probing privacy leakage in large language models," in *Proc. Adv. Neural Inf. Process. Syst.: Annu. Conf. Neural Inf. Process. Syst.*, New Orleans, LA, USA, 2023, pp. 20750–20762.
[18] J. Shao et al., "A survey of what to share in federated learning: Perspectives on model utility, privacy leakage, and communication efficiency," 2023, *arXiv:2307.10655*.
[19] G. Xu, S. Xu, J. Ning, X. Huang, H. Li, and R. Lu, "New secure sparse inner product with applications to machine learning," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 4, pp. 4040–4052, Jul./Aug. 2025.
[20] Y. Lai et al., "Two-phase defense against poisoning attacks on federated learning-based intrusion detection," *Comput. Secur.*, vol. 129, 2023, Art. no. 103205.
[21] X. Li, Z. Qu, S. Zhao, B. Tang, Z. Lu, and Y. Liu, "LoMar: A local defense against poisoning attack on federated learning," *IEEE Trans. Dependable Secur. Comput.*, vol. 20, no. 1, pp. 437–450, Jan./Feb. 2023.
[22] B. Wang, H. Li, Y. Guo, and J. Wang, "PPFLHE: A privacy-preserving federated learning scheme with homomorphic encryption for healthcare data," *Appl. Soft Comput.*, vol. 146, 2023, Art. no. 110677.
[23] S. M. Hosseini, M. Sikaroudi, M. Babaie, and H. R. Tizhoosh, "Cluster based secure multi-party computation in federated learning for histopathology images," 2022, *arXiv:2208.10919*.
[24] K. Wei et al., "Personalized federated learning with differential privacy and convergence guarantee," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 4488–4503, 2023.
[25] Y. Sun, Y. Mao, and J. Zhang, "MimiC: Combating client dropouts in federated learning by mimicking central updates," *IEEE Trans. Mobile Comput.*, vol. 23, no. 7, pp. 7572–7584, Jul. 2024.
[26] H. Chen, H. Zhao, Z. Zhang, and K. Li, "Discriminative feature learning-based federated lightweight distillation against multiple attacks," *IEEE Internet Things J.*, vol. 11, no. 10, pp. 17663–17677, May 2024.
[27] F. Luo, S. Al-Kuwari, and Y. Ding, "SVFL: Efficient secure aggregation and verification for cross-silo federated learning," *IEEE Trans. Mobile Comput.*, vol. 23, no. 1, pp. 850–864, Jan. 2024.
[28] J. Fan and F. Vercauteren, "Somewhat practical fully homomorphic encryption," *IACR Cryptol. ePrint Arch.*, vol. 2022, 2012, Art. no. 144.
[29] J. H. Bell, K. A. Bonawitz, A. Gascón, T. Lepoint, and M. Raykova, "Secure single-server aggregation with (poly)logarithmic overhead," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, USA, 2020, pp. 1253–1269.
[30] G. Xu, H. Li, S. Liu, K. Yang, and X. Lin, "VerifyNet: Secure and verifiable federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 911–926, 2020.
[31] A. Kim, Y. Song, M. Kim, K. Lee, and J. H. Cheon, "Logistic regression model training based on the approximate homomorphic encryption," *IACR Cryptol. ePrint Arch.*, vol. 2018, 2018, Art. no. 254.
[32] L. T. Phong, Y. Aono, T. Hayashi, L. Wang, and S. Moriai, "Privacy-preserving deep learning via additively homomorphic encryption," *IEEE Trans. Inf. Forensics Secur.*, vol. 13, no. 5, pp. 1333–1345, May 2018.
[33] S. Truex et al., "A hybrid approach to privacy-preserving federated learning - (extended abstract)," *Inform. Spektrum*, vol. 42, no. 5, pp. 356–357, 2019.
[34] P. Mohassel and Y. Zhang, "SecureML: A system for scalable privacy-preserving machine learning," in *Proc. IEEE Symp. Secur. Privacy*, San Jose, CA, USA, 2017, pp. 19–38.
[35] H. Chaudhari, R. Rachuri, and A. Suresh, "Trident: Efficient 4PC framework for privacy preserving machine learning," in *Proc. 27th Annu. Netw. Distrib. Syst. Secur. Symp.*, San Diego, California, USA, 2020.
[36] K. A. Bonawitz et al., "Practical secure aggregation for federated learning on user-held data," 2016, *arXiv:1611.04482*.
[37] Y. Liu, Z. Ma, Z. Yan, Z. Wang, X. Liu, and J. Ma, "Privacy-preserving federated k-means for proactive caching in next generation cellular networks," *Inf. Sci.*, vol. 521, pp. 14–31, 2020.
[38] Z. Liu, J. Guo, K. Lam, and J. Zhao, "Efficient dropout-resilient aggregation for privacy-preserving machine learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 1839–1854, 2023.
[39] R. Fotohi, F. S. Aliee, and B. J. Farahani, "Decentralized and robust privacy-preserving model using blockchain-enabled federated deep learning in intelligent enterprises," *Appl. Soft Comput.*, vol. 161, 2024, Art. no. 111764.
[40] R. Fotohi, F. S. Aliee, and B. J. Farahani, "A lightweight and secure deep learning model for privacy-preserving federated learning in intelligent enterprises," *IEEE Internet Things J.*, vol. 11, no. 19, pp. 31988–31998, Oct. 2024.
[41] Y. Ren, Y. Li, G. Feng, and X. Zhang, "Privacy-enhanced and verification-traceable aggregation for federated learning," *IEEE Internet Things J.*, vol. 9, no. 24, pp. 24933–24948, Dec. 2022.
[42] J. Konečný, H. B. McMahan, D. Ramage, and P. Richtárik, "Federated optimization: Distributed machine learning for on-device intelligence," 2016, *arXiv:1610.02527*.
[43] A. Shamir, "How to share a secret," *Commun. ACM*, vol. 22, no. 11, pp. 612–613, 1979.



Jiayin Li received the PhD degree in computer science from Fuzhou University, Fuzhou, China. He is currently a teacher with the College of Computer and Cyber Security, Fujian Normal University. His research interests include data security, artificial intelligence security, autonomous driving security, and anomaly detection. He has published papers on intelligent information processing and privacy-preserving, including those in *IEEE Transactions on Vehicular Technology*, *IEEE Transactions on Network Science and Engineering*, *IEEE Transactions on Dependable and Secure Computing*, ACM KDD, and IEEE Globecom.

Authorized licensed use limited to: Anhui University. Downloaded on October 13,2026 at 09:40:23 UTC from IEEE Xplore. Restrictions apply.



Shengmin Xu (Member, IEEE) is currently a full professor with the College of Computer and Cyber Security, Fujian Normal University, Fuzhou, China. Previously, he served as a senior research engineer with Singapore Management University. He has published more than 50 research papers in refereed international conferences and journals, such as IEEE S&P, ESORICS, ACSAC, AsiaCCS, *IEEE Information Forensics & Security*, and *IEEE Transactions on Dependable & Secure Computing*. His research interests include cryptography and information security. He is on the editorial boards of *IEEE Transactions on Dependable & Secure Computing* and *IEEE Transactions on Circuits & Systems for Video Technology*.



Xingshuo Han received the PhD degree from the College of Computing and Data Science, Nanyang Technological University. He is a professor with the College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics. He has published papers in reputable security and AI conferences and journals, including IEEE S&P, ACM CCS, Usenix Security, *IEEE Transactions on Dependable and Secure Computing*, ISSTA, NeurIPS, and ICCV. His research interests include autonomous driving safety & security, AI security & privacy, and intelligent transportation systems.



Jianting Ning (Member, IEEE) received the PhD degree from the Department of Computer Science and Engineering, Shanghai Jiao Tong University, in 2016. He is with the School of Cyber Science and Engineering, Wuhan University, Wuhan, China, and also with the Faculty of Data Science, City University of Macau, Macau, China. He has published papers in major conferences or journals such as ACM CCS, NDSS, Asiacrypt, ESORICS, ACSAC, *IEEE Transactions on Information Forensics and Security*, and *IEEE Transactions on Dependable and Secure Computing*. His research interests include applied cryptography and information security.



Xinlei He (Associate Member, IEEE) received the PhD degree from CISPA Helmholtz Center for Information Security. He is an assistant professor with the DSA & IoT Thrust, Information Hub, HKUST(GZ). His research include domain of trustworthy machine learning, with a special focus on privacy, security, and accountability issues stemming from machine learning paradigms. He has published more than 30 papers in top-tier conferences/journals such as IEEE S&P, ACM CCS, USENIX Security, and NDSS. He served as the TPC member of IEEE S&P, IEEE Euro S&P, ASIACCS, ACSAC, IEEE ICDCS, and ESORICS. He has received the Norton Graduate Fellowship 2022 (2 people around the world), the Best Paper Award at LAMPS'24, and the Distinguished Poster Award at NDSS'25.



Guowen Xu (Senior Member, IEEE) received the PhD degree in cyberspace security from the University of Electronic Science and Technology of China. He is a full professor with the University of Electronic Science and Technology of China (UESTC). His research include cybersecurity, AI security and privacy, and applied cryptography. Before joining UESTC, he has held positions as a research fellow with Nanyang Technological University, Singapore, and a senior research fellow with the City University of Hong Kong. He has published more than 100 papers in top-tier IEEE journals and conferences, including *IEEE Transactions on Dependable and Secure Computing (TDSC)*, *IEEE Transactions on Information Forensics and Security (TIFS)*, IEEE INFOCOM, IEEE S&P, ICML, and NeurIPS. Hiswork has been recognized with multiple awards, including the IEEE BigDataSecurity Best Paper Award (2023), IEEE ICPADS Best Paper Award (2020), Wu Wenjun First Prize of Artificial Intelligence Science and Technology Progress (2021), IEEE Early Career Speaker, IEEE Computer Society (2025), and Computing's Top 30 Early Career Professionals, IEEE Computer Society (2025). He has extensive editorial experience, currently serving as an associate editor for *IEEE Transactions on Dependable and Secure Computing (TDSC)*, *IEEE Transactions on Information Forensics and Security (TIFS)*, *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, *IEEE Transactions on Circuits and Systems for Video Technology*, and *IEEE Transactions on Network and Service Management*. He is also a lead guest editor for *ACM Transactions on Autonomous and Adaptive Systems (TAAS)*. Beyond his editorial roles, he has been an area chair or senior program committee member for prestigious international conferences, including ICML, ICLR, KDD, AAAI, NeurIPS, and CSCW.

Authorized licensed use limited to: Anhui University. Downloaded on October 13,2026 at 09:40:23 UTC from IEEE Xplore. Restrictions apply.